Analysis3

March 3, 2026

1 Physics 332 Analysis Assignment 3

Ahilan

we ask, if I repeatedly generate noisy data from a known true line and fit it each time, what distribution do I get for my estimated parameters?

[14]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
```

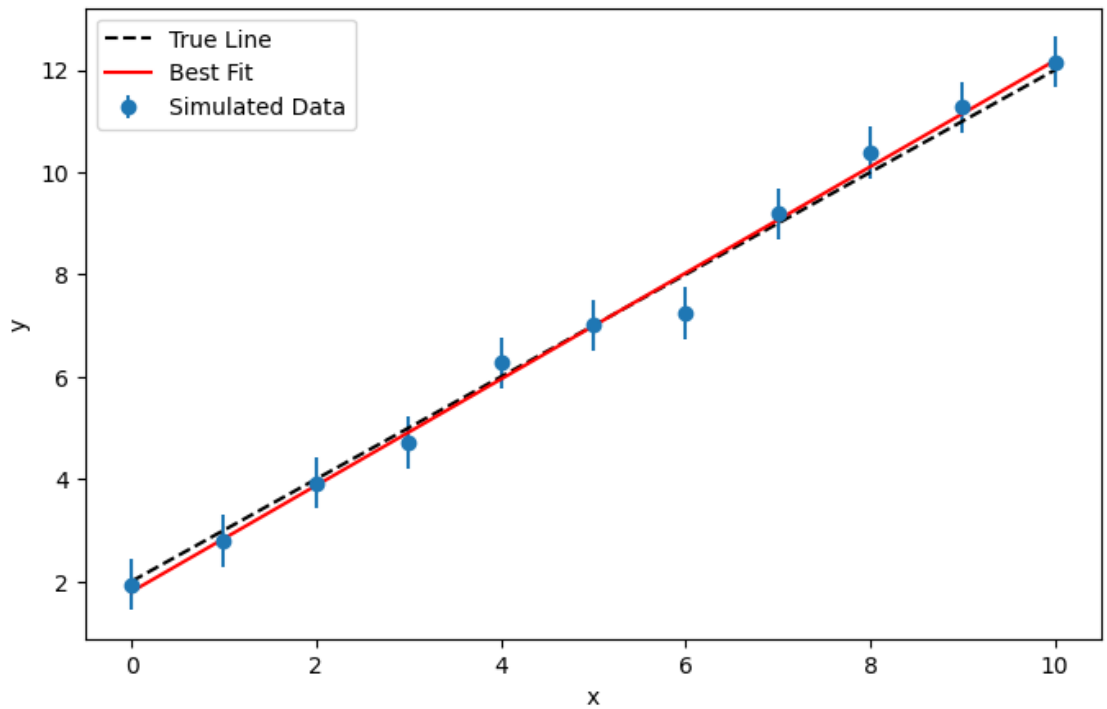
1.1 1. Monte Carlo simulation of fitting performance (20 points)

(a)

[15]:
```
# Question 1 Part (a)
true_a = 2.0
true_b = 1.0
sigma_noise = 0.5 # The Question says its centered on 0

def linear_model(x, a, b):
    return a + b * x

def generate_and_fit(N=11, a=true_a, b=true_b, sigma=sigma_noise,␣
↪return_data=False):
    # Setup numpy default rng (As seen in Analysis Assignment 1)
    rng = np.random.default_rng() # Random Generator function

    x = np.arange(N) # Number 0 to 10

    noise = rng.normal(0.0, sigma, N) # Normal or Gaussian Noise

    y = a + b * x + noise

    popt, pcov = curve_fit(linear_model, x, y, sigma=np.full(N, sigma),␣
↪absolute_sigma=True)

    a_est, b_est = popt # Estimates
    a_err, b_err = np.sqrt(np.diag(pcov))
```

```python
    covariance = pcov[0, 1]
    correlation = covariance / (a_err * b_err)

    if return_data: # Just an Argument I made so that I can opt to return the␣
 ↪Random Data points as well
        return x, y, a_est, b_est, a_err, b_err, correlation

    return a_est, b_est, a_err, b_err, correlation

x_sample, y_sample, a_est, b_est, a_err, b_err, corr =␣
 ↪generate_and_fit(return_data=True)

print(f"Results from a single trial (N=11):")
print(f"Estimated intercept a: {a_est:.4f} +/- {a_err:.4f}")
print(f"Estimated slope b: {b_est:.4f} +/- {b_err:.4f}")
print(f"Correlation coefficient: {corr:.4f}")

plt.figure(figsize=(8, 5))
plt.errorbar(x_sample, y_sample, yerr=sigma_noise, fmt='o', label='Simulated␣
 ↪Data')
plt.plot(x_sample, linear_model(x_sample, true_a, true_b), 'k--', label='True␣
 ↪Line')
plt.plot(x_sample, linear_model(x_sample, a_est, b_est), 'r-', label='Best Fit')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()
```

```
Results from a single trial (N=11):
Estimated intercept a: 1.7974 +/- 0.2820
Estimated slope b: 1.0388 +/- 0.0477
Correlation coefficient: -0.8452
```

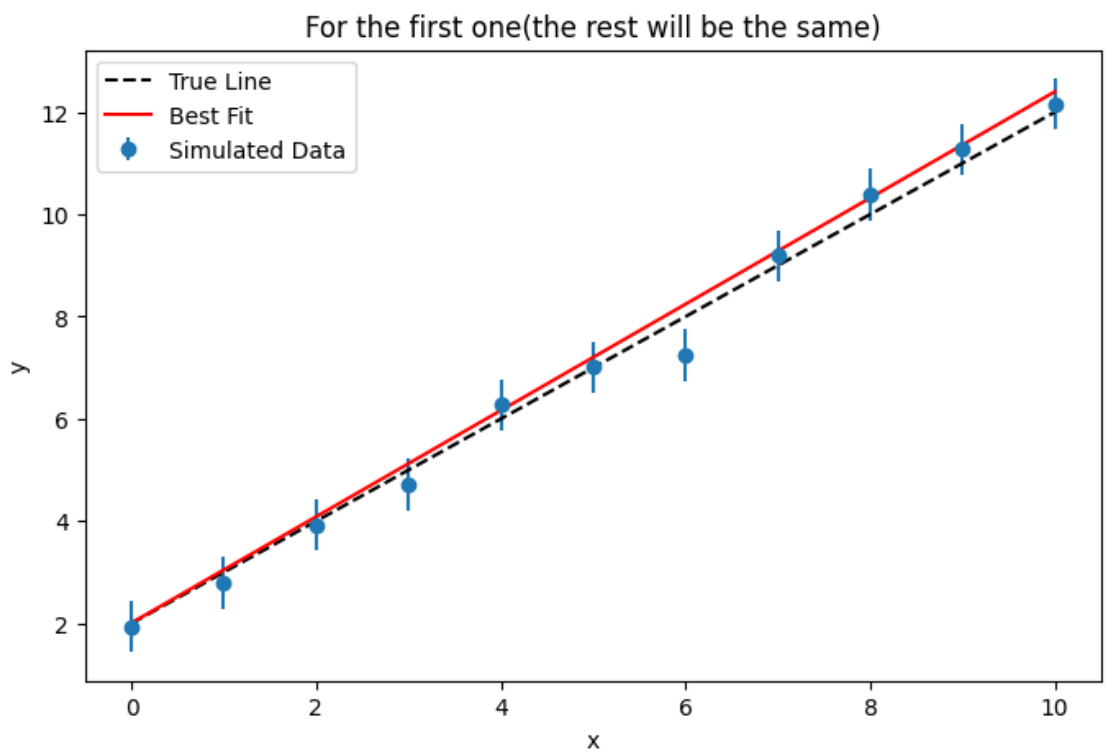


(b)

[24]:
```
# Question 1 Part (b)
N_sim = 1000

a_array = np.zeros(N_sim)
b_array = np.zeros(N_sim)
a_err_theory = 0.0
b_err_theory = 0.0

for i in range(N_sim):
    a_est, b_est, a_err, b_err, corr = generate_and_fit()
    a_array[i] = a_est
    b_array[i] = b_est
    if i == 0:
        a_err_theory = a_err
        b_err_theory = b_err
        corr_theory = corr


print(f"Results from a single trial (N={N_sim}):")
print(f"Estimated intercept a: {a_est:.4f} +/- {a_err:.4f}")
print(f"Estimated slope b: {b_est:.4f} +/- {b_err:.4f}")
print(f"Correlation coefficient: {corr:.4f}")
```

```
plt.figure(figsize=(8, 5))
plt.errorbar(x_sample, y_sample, yerr=sigma_noise, fmt='o', label='Simulated␣
↪Data')
plt.plot(x_sample, linear_model(x_sample, true_a, true_b), 'k--', label='True␣
↪Line')
plt.plot(x_sample, linear_model(x_sample, a_est, b_est), 'r-', label='Best Fit')
plt.xlabel('x')
plt.ylabel('y')
plt.title("For the first one(the rest will be the same)")
plt.legend()
plt.show()
```

```
Results from a single trial (N=1000):
Estimated intercept a: 2.0087 +/- 0.2820
Estimated slope b: 1.0390 +/- 0.0477
Correlation coefficient: -0.8452
```



(c)

[17]:
```
# Question 1 Part (c)
a_mean = np.mean(a_array) # Mean a
b_mean = np.mean(b_array) # Mean b
```

```
a_std = np.std(a_array, ddof=1) # Std A
b_std = np.std(b_array, ddof=1) # Std B

a_mean_err = a_std / np.sqrt(N_sim) # Std (A's Mean)
b_mean_err = b_std / np.sqrt(N_sim) # Std (B's Mean)


mc_correlation = np.corrcoef(a_array, b_array)[0, 1]


print("1. Mean Values:")
print(f"  Mean a = {a_mean:.4f} +/- {a_mean_err:.4f}  (True value: {true_a})")
print(f"  Mean b = {b_mean:.4f} +/- {b_mean_err:.4f}  (True value: {true_b})")
print("\n2. Parameter Uncertainties (Standard Deviations):")
print(f"  MC Std. Dev. of a = {a_std:.4f}  (Theoretical from curve_fit:␣
↪{a_err_theory:.4f})")
print(f"  MC Std. Dev. of b = {b_std:.4f}  (Theoretical from curve_fit:␣
↪{b_err_theory:.4f})")
print("\n3. Parameter Correlation:")
print(f"  MC Correlation coefficient = {mc_correlation:.4f}  (Theoretical from␣
↪curve_fit: {corr_theory:.4f})")
```

```
1. Mean Values:
  Mean a = 1.9908 +/- 0.0093  (True value: 2.0)
  Mean b = 1.0014 +/- 0.0015  (True value: 1.0)

2. Parameter Uncertainties (Standard Deviations):
  MC Std. Dev. of a = 0.2934  (Theoretical from curve_fit: 0.2820)
  MC Std. Dev. of b = 0.0485  (Theoretical from curve_fit: 0.0477)

3. Parameter Correlation:
  MC Correlation coefficient = -0.8596  (Theoretical from curve_fit: -0.8452)
```

Conclusion: From the Above, I can see how they are very close!!! So yes, the Monte Carlo Simualtion works.

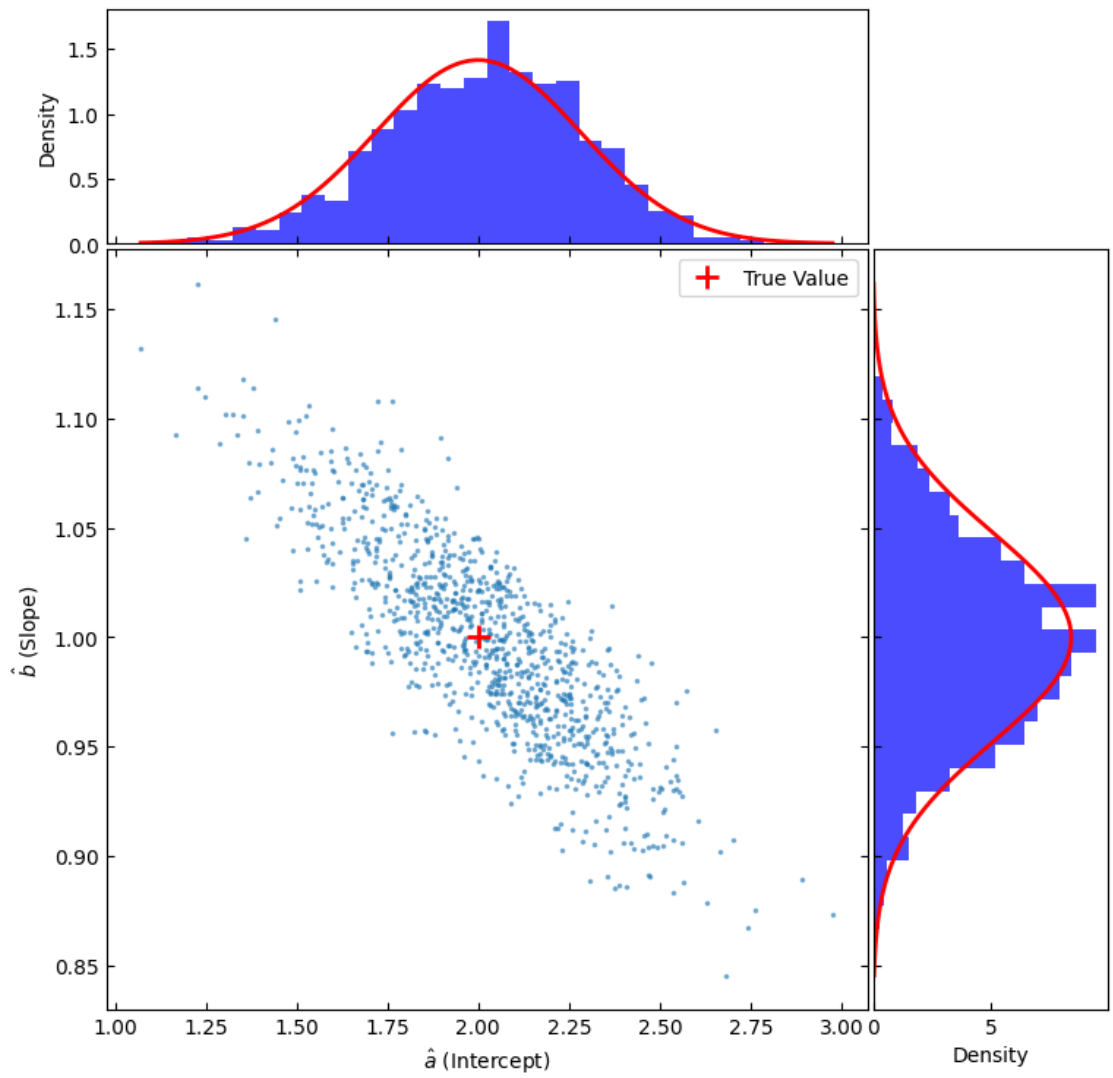
(d)

[25]:
```
# (Note: Help from AI to help me code and Plot this part)
# Question 1 Part (d)

left, width = 0.1, 0.65
bottom, height = 0.1, 0.65
spacing = 0.005

fig = plt.figure(figsize=(8, 8))
```

```
ax_scatter = plt.axes([left, bottom, width, height])
ax_scatter.tick_params(direction='in', top=True, right=True)
ax_histx = plt.axes([left, bottom + height + spacing, width, 0.2])
ax_histx.tick_params(direction='in', labelbottom=False)
ax_histy = plt.axes([left + width + spacing, bottom, 0.2, height])
ax_histy.tick_params(direction='in', labelleft=False)

ax_scatter.scatter(a_array, b_array, alpha=0.5, marker='.', s=10)
ax_scatter.set_xlabel('$\hat{a}$ (Intercept)')
ax_scatter.set_ylabel('$\hat{b}$ (Slope)')
ax_scatter.plot(true_a, true_b, 'r+', markersize=12, markeredgewidth=2,␣
 ↪label='True Value')
ax_scatter.legend()

binwidth_a = (np.max(a_array) - np.min(a_array)) / 30
binwidth_b = (np.max(b_array) - np.min(b_array)) / 30
bins_a = np.arange(np.min(a_array), np.max(a_array) + binwidth_a, binwidth_a)
bins_b = np.arange(np.min(b_array), np.max(b_array) + binwidth_b, binwidth_b)

ax_histx.hist(a_array, bins=bins_a, density=True, color='blue', alpha=0.7)
ax_histy.hist(b_array, bins=bins_b, density=True, orientation='horizontal',␣
 ↪color='blue', alpha=0.7)

x_a = np.linspace(bins_a[0], bins_a[-1], 100)
gaussian_a = (1 / (a_err_theory * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((x_a -␣
 ↪true_a) / a_err_theory)**2)
ax_histx.plot(x_a, gaussian_a, 'r-', lw=2)

x_b = np.linspace(bins_b[0], bins_b[-1], 100)
gaussian_b = (1 / (b_err_theory * np.sqrt(2 * np.pi))) * np.exp(-0.5 * ((x_b -␣
 ↪true_b) / b_err_theory)**2)
ax_histy.plot(gaussian_b, x_b, 'r-', lw=2)

ax_histx.set_xlim(ax_scatter.get_xlim())
ax_histy.set_ylim(ax_scatter.get_ylim())
ax_histx.set_ylabel('Density')
ax_histy.set_xlabel('Density')

plt.show()
```

```
<>:18: SyntaxWarning: invalid escape sequence '\h'
<>:19: SyntaxWarning: invalid escape sequence '\h'
<>:18: SyntaxWarning: invalid escape sequence '\h'
<>:19: SyntaxWarning: invalid escape sequence '\h'
C:\Users\ahila\AppData\Local\Temp\ipykernel_46996\2301686980.py:18:
SyntaxWarning: invalid escape sequence '\h'
  ax_scatter.set_xlabel('$\hat{a}$ (Intercept)')
```

```
C:\Users\ahila\AppData\Local\Temp\ipykernel_46996\2301686980.py:19:
SyntaxWarning: invalid escape sequence '\h'
  ax_scatter.set_ylabel('$\hat{b}$ (Slope)')
```



Conclusion: I can Clealry see the Negative corrlation (between slope and intercept).

1.2 2. Bootstrap estimation of parameter uncertainties (BONUS 10 points)

[27]:
```
# Question 2 Parts (a) and (b)
N_data2 = 20
rng = np.random.default_rng(seed=10)
x_data = np.arange(N_data2)
noise2 = rng.normal(0.0, sigma_noise, N_data2)
y_data = true_a + true_b * x_data + noise2
```

```
popt2, pcov2 = curve_fit(linear_model, x_data, y_data, sigma=np.full(N_data2,␣
↪sigma_noise), absolute_sigma=True)
a_fit, b_fit = popt2
a_fit_err, b_fit_err = np.sqrt(np.diag(pcov2))

print("For Base Fit Results Data Set (We are trying to aim for)")
print(f"Fitted intercept: a = {a_fit:.4f} +/- {a_fit_err:.4f}")
print(f"Fitted slope:     b = {b_fit:.4f} +/- {b_fit_err:.4f}")
```

```
For Base Fit Results Data Set (We are trying to aim for)
Fitted intercept: a = 1.8946 +/- 0.2155
Fitted slope:     b = 0.9950 +/- 0.0194
```

(c)

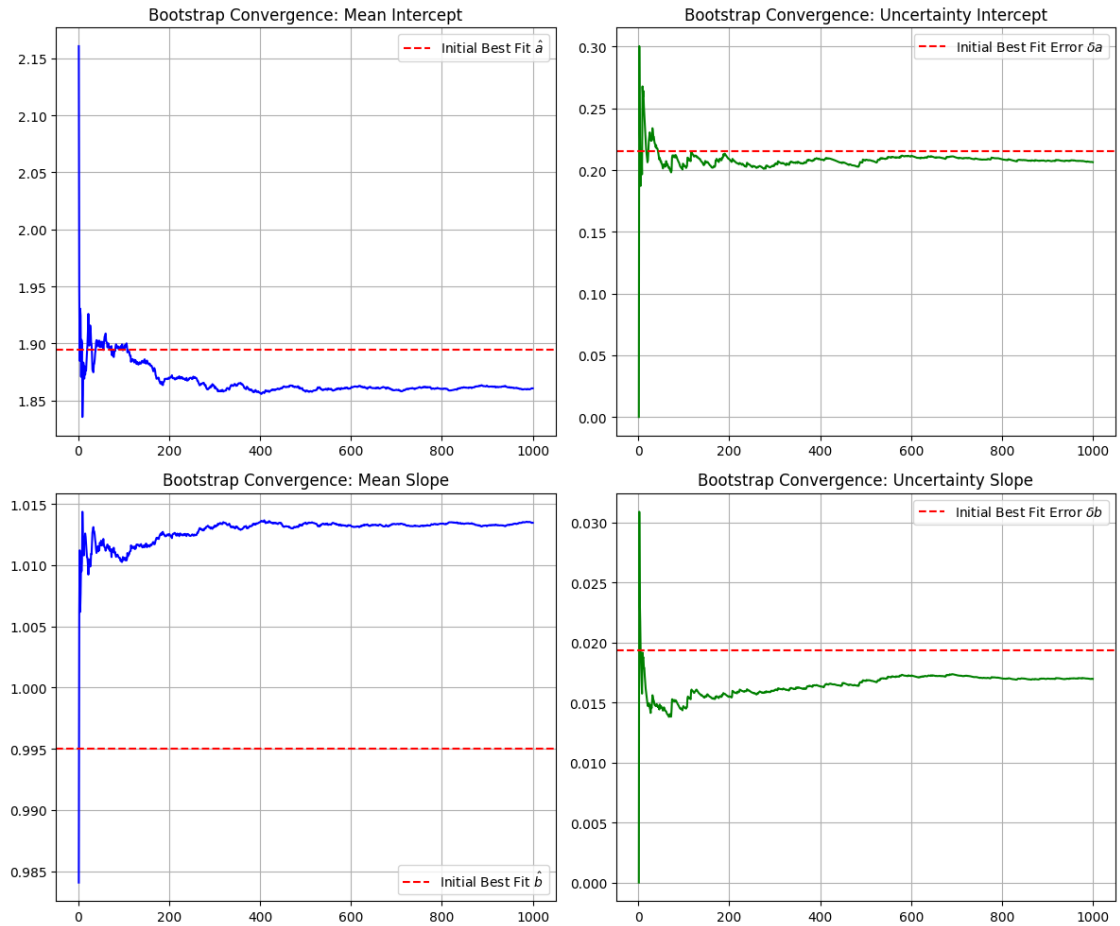
[ ]:
```
# Question 2 Part (c)
N_boot = 1000
a_boot = np.zeros(N_boot)
b_boot = np.zeros(N_boot)
a_cum_mean = np.zeros(N_boot)
b_cum_mean = np.zeros(N_boot)
a_cum_std = np.zeros(N_boot)
b_cum_std = np.zeros(N_boot)

for i in range(N_boot): # Interate
    # Using rng.choice to sample with replacement, just found this
    indices = rng.choice(N_data2, N_data2, replace=True)
    x_syn = x_data[indices]
    y_syn = y_data[indices]

    # Here we fit the line to the Self geenrated Data
    popt_syn, _ = curve_fit(linear_model, x_syn, y_syn)
    a_boot[i] = popt_syn[0] # Storing it respectively
    b_boot[i] = popt_syn[1]

    a_cum_mean[i] = np.mean(a_boot[:i+1])
    b_cum_mean[i] = np.mean(b_boot[:i+1])
    if i == 0:
        a_cum_std[i] = 0.0
        b_cum_std[i] = 0.0
    else:
        a_cum_std[i] = np.std(a_boot[:i+1], ddof=1)
        b_cum_std[i] = np.std(b_boot[:i+1], ddof=1)
```

(d)

[ ]:
```
# Question 2 Part (d)
# Help from AI to help me plot this.

iterations = np.arange(1, N_boot + 1)
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# Subplots code structure
axs[0, 0].plot(iterations, a_cum_mean, 'b-')
axs[0, 0].axhline(a_fit, color='r', linestyle='--', label='Initial Best Fit␣
↪$\hat{a}$')
axs[0, 0].set_title('Bootstrap Convergence: Mean Intercept')
axs[0, 0].legend()


axs[0, 1].plot(iterations, a_cum_std, 'g-')
axs[0, 1].axhline(a_fit_err, color='r', linestyle='--', label='Initial Best Fit␣
↪Error $\delta a$')
axs[0, 1].set_title('Bootstrap Convergence: Uncertainty Intercept')
axs[0, 1].legend()


axs[1, 0].plot(iterations, b_cum_mean, 'b-')
axs[1, 0].axhline(b_fit, color='r', linestyle='--', label='Initial Best Fit␣
↪$\hat{b}$')
axs[1, 0].set_title('Bootstrap Convergence: Mean Slope')
axs[1, 0].legend()


axs[1, 1].plot(iterations, b_cum_std, 'g-')
axs[1, 1].axhline(b_fit_err, color='r', linestyle='--', label='Initial Best Fit␣
↪Error $\delta b$')
axs[1, 1].set_title('Bootstrap Convergence: Uncertainty Slope')
axs[1, 1].legend()

plt.tight_layout()
plt.show()
```

```
<>:9: SyntaxWarning: invalid escape sequence '\h'
<>:15: SyntaxWarning: invalid escape sequence '\d'
<>:21: SyntaxWarning: invalid escape sequence '\h'
<>:27: SyntaxWarning: invalid escape sequence '\d'
<>:9: SyntaxWarning: invalid escape sequence '\h'
<>:15: SyntaxWarning: invalid escape sequence '\d'
<>:21: SyntaxWarning: invalid escape sequence '\h'
<>:27: SyntaxWarning: invalid escape sequence '\d'
C:\Users\ahila\AppData\Local\Temp\ipykernel_46996\802239035.py:9: SyntaxWarning:
invalid escape sequence '\h'
```

```
  axs[0, 0].axhline(a_fit, color='r', linestyle='--', label='Initial Best Fit
$\hat{a}$')
C:\Users\ahila\AppData\Local\Temp\ipykernel_46996\802239035.py:15:
SyntaxWarning: invalid escape sequence '\d'
  axs[0, 1].axhline(a_fit_err, color='r', linestyle='--', label='Initial Best
Fit Error $\delta a$')
C:\Users\ahila\AppData\Local\Temp\ipykernel_46996\802239035.py:21:
SyntaxWarning: invalid escape sequence '\h'
  axs[1, 0].axhline(b_fit, color='r', linestyle='--', label='Initial Best Fit
$\hat{b}$')
C:\Users\ahila\AppData\Local\Temp\ipykernel_46996\802239035.py:27:
SyntaxWarning: invalid escape sequence '\d'
  axs[1, 1].axhline(b_fit_err, color='r', linestyle='--', label='Initial Best
Fit Error $\delta b$')
```



3/4 Seem to converge really well.

(e)

[22]:
```
# Question 2 Part (e)
print(f"=== Comparison Configuration (Data Set N={N_data2}, Bootstrap␣
↪Iterations={N_boot}) ===")
print("Initial Weighted Fit Statistics:")
print(f"Intercept: {a_fit:.4f} +/- {a_fit_err:.4f}")
print(f"Slope:     {b_fit:.4f} +/- {b_fit_err:.4f}")

print("\nBootstrap Statistics (Cumulative at Iteration 1000):")
print(f"Intercept: {a_cum_mean[-1]:.4f} +/- {a_cum_std[-1]:.4f}")
print(f"Slope:     {b_cum_mean[-1]:.4f} +/- {b_cum_std[-1]:.4f}")
```

```
=== Comparison Configuration (Data Set N=20, Bootstrap Iterations=1000) ===
Initial Weighted Fit Statistics:
Intercept: 1.8630 +/- 0.2155
Slope:     1.0127 +/- 0.0194

Bootstrap Statistics (Cumulative at Iteration 1000):
Intercept: 1.8606 +/- 0.2064
Slope:     1.0135 +/- 0.0170
```

Conclusion: Again they Match really really well ! :)